

Reviewer Pack — Arithmetic Hierarchy Invariant Bounds

Frege Proof Depth

Entry 48c214d28fcc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 18:20:57 UTC

Arithmetic Hierarchy Invariant Bounds Frege Proof Depth

Entry ID: 48c214d28fcc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 18:20:57 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Hierarchy Theory **Field B** (complexity object): Frege Proof Complexity

Statement:

For any given Tseitin formula π with n variables, the Frege proof depth of π is bounded by the length of the longest sequence of jumps in the arithmetic hierarchy that can be represented using natural numbers derived from π , i.e., $f(\pi) \leq L(\pi)$, where $f(\pi)$ represents the Frege proof depth and $L(\pi)$ is a function of n .

Rationale (proposer's reasoning):

The arithmetic hierarchy provides a framework for classifying computational problems by their difficulty. If this conjecture holds, it would suggest a deep connection between the structure of arithmetic hierarchies and the complexity of proving satisfiability using Frege's method. This could potentially lead to new insights into the nature of NP-completeness.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2977d6715463441c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all Tseitin formulas π with n variables, the Frege proof depth $f(\pi)$ is less than or equal to $L(\pi)$, where $L(\pi)$ is computed from π and measured across at least 30 seeds. The criterion is falsified if any seed has a Frege proof depth $f(\pi)$ greater than $L(\pi)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Arithmetic Hierarchy" AND "Frege proof complexity" - Invariant bounds ON "Tseitin formula" AND Frege proof depth - "Proof depth" IN arithmetic hierarchy AND Tseitin formula

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=246.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_tseitin_formula(n):
    if n == 1:
        return "A"
    else:
        A = "A" + str(n)
        B = "B" + str(n)
        C = "C" + str(n)
        formula = f"({A} & {B}) | ({B} & {C})"
        for i in range(1, n):
            sub_formula = generate_tseitin_formula(i)
            formula = f"({formula} & {sub_formula})"
        return formula

def frege_proof_depth(formula):
    if formula == "A":
        return 1
    elif formula.startswith("(") and formula.endswith(")"):
        parts = formula[1:-1].split("&")
        max_depth = 0
        for part in parts:
            depth = frege_proof_depth(part)
            if depth > max_depth:
                max_depth = depth
        return 1 + max_depth
    else:
        return float('inf')

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        formula = generate_tseitin_formula(n)
        depth = frege_proof_depth(formula)
        L_pi = len(formula.split("&"))
        if depth > L_pi:
            conjecture_holds = False
            counterexample = f"Formula: {formula}, Depth: {depth}, L(pi): {L_pi}"
            break
        total_metric_value += depth
        instances_tested += 1
    n_max = max(n_max, n)

return {
    "metric_name": "Frege Proof Depth",
    "metric_value": total_metric_value / instances_tested,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14861
2	preregistration	ollama_remote	glm4:latest	0	0	9290
3	novelty	ollama_remote	glm4:latest	0	0	8334
4	novelty	ollama_remote	glm4:latest	0	0	8815
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11352
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22331
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23299
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15700
9	judge	ollama_remote	glm4:latest	0	0	113559

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 227541 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/48c214d28fcc.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/48c214d28fcc.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/48c214d28fcc.tar.gz* (if generated)